CSE 220 · SIGNALS AND LINEAR SYSTEMS

NumPy & Matplotlib Lab Cheatsheet

Everything needed for the discrete-signal onlines: building signals, reversal, shifting, scaling, even-odd decomposition, energy/power, sinusoids, system-property tests, and plotting.

Convention used throughout: `INF = 8`, a signal is a length-17 array on `n = -8 …8`, and anything outside that window is `0`.

Contents

1 Axes & Building Signals

Every task starts by making an index/time axis, then applying math elementwise.

```
import numpy as np

n = np.arange(-8, 9)          # integer indices -8..8 (stop excluded)
t = np.linspace(0, np.pi, 500)  # 500 points, endpoint INCLUDED

x = np.zeros(2*8+1)           # blank length-17 signal
x = np.zeros_like(t)          # zeros, same shape/dtype as t

x = np.sin(t)                 # ufuncs broadcast over whole array
x = np.cos(2*np.pi*5*t)         # 5 Hz cosine
x = np.abs(x)**2              # elementwise magnitude squared
```

Tip

arange vs linspace: `arange(a,b,step)` excludes the stop and is for integer indices; `linspace(a,b,num)` includes the endpoint and is for a fixed number of "continuous" points.

Piecewise signals with boolean masks

```
t = np.linspace(-4, 4, 4001)
x = np.zeros_like(t)
m = np.abs(t) <= 1.0            # boolean mask
x[m] = 1.0 - np.abs(t[m])     # triangular pulse on |t|<=1
```

2 Time Reversal — $x[-n], x(-t)$

```
x_rev = x[::-1]          # reverse the values
x_rev = np.flip(x)        # same thing, explicit
n_rev = -n[::-1]         # MUST also flip + negate the axis
```

Trap / Warning

The #1 mistake: `x[::-1]` reverses values but the index axis must flip too (a value at $n = 3$ moves to $n = -3$). Plain slicing is only correct when the axis is **symmetric about 0** — which it is for $n = -8 \ldots 8$, so `x[::-1]` is fine in the onlines. On an asymmetric axis it silently corrupts every label.

3 Time Shift — $x[n - k]$

Positive k delays (shifts right); negative k advances (left). Minus-in-the-argument = right.

```python
def time_shift_signal(x, k):
    if k > 0:       # delay: zeros in from the left, tail drops off
        return np.concatenate((np.zeros(k), x[:-k]))
    elif k < 0:     # advance
        return np.concatenate((x[-k:], np.zeros(-k)))
    return x.copy()
```

Trap / Warning

Do NOT use `np.roll` for a real delay. `np.roll(x,k)` is *circular* — values that fall off one end reappear at the other. Only correct for periodic signals, never for a true shift.

4 Time Scaling — $x[kn]$ and $x[n/k]$

Compression $x[kn]$ (positive integer k)

```python
def time_scale_signal(x, k):          # y[n] = x[kn]
    t = np.arange(-8, 9) * k      # argument kn for each output n
    ans = np.zeros_like(x)
    mask = (t >= -8) & (t <= 8)  # keep args inside window
    ans[mask] = x[t[mask] + 8]       # +8 maps index -> array position
    return ans
```

Expansion $x[n/k]$ — zeros between (Task-1 style)

```python
def time_scale_signal(x, k):          # y[n] = x[n/k]
    n = np.arange(-8, 9)
    m = n / k
    is_exact = (m == np.round(m))    # only integer n/k are real
        samples
    m_int = np.round(m).astype(int)
    ans = np.zeros_like(x)
    ans[is_exact] = x[m_int[is_exact] + 8]
    return ans
```

Expansion $x[n/k]$ — with averaging (Task-2 style)

```
def time_scale_signal_interpolate(x, k):
    n = np.arange(-8, 9)
    m = n / k
    m_lo = np.floor(m).astype(int)
    m_hi = np.ceil(m).astype(int)
    def sample(idx):
        v = np.zeros(len(idx))
        ok = (idx >= -8) & (idx <= 8)
        v[ok] = x[idx[ok] + 8]
        return v
    lo, hi = sample(m_lo), sample(m_hi)
    return np.where(m_lo == m_hi, lo, (lo + hi) / 2)
```

Tip

Why the `m_lo == m_hi` check: when n/k is exactly an integer, floor and ceil are equal — return the true sample, don't average it with itself (harmless here but essential when $k = 1$ must be identity).

5 General Index Lookup $x[\alpha n + \beta]$

One function for shift + reversal + scaling together, with zero-fill. Evaluate x at the transformed argument.

```
def transform(n_samp, x_samp, alpha, beta, n_out):
    arg = alpha * n_out + beta     # where to read from, per output n
    out = np.zeros(n_out.shape)
    is_int = (arg == np.round(arg))  # discrete: integer args only
    ai = np.round(arg).astype(int)
    pos = np.searchsorted(n_samp, ai) # candidate slot (binary search)
    pos = np.clip(pos, 0, len(n_samp)-1) # keep index legal
    valid = is_int & (n_samp[pos] == ai) # confirm real match
    out[valid] = x_samp[pos[valid]]  # copy hits; rest stay 0
    return out
```

Tip

`searchsorted` finds where a value *would* go, not whether it's there. Always pair it with the `n_samp[pos] == ai` check. `clip` is just a crash-guard so the check can run.

Continuous version (interpolate between samples)

```
def transform_cont(t, x, alpha, beta, t_query):
    return np.interp(alpha*t_query + beta, t, x,
                     left=0.0, right=0.0)  # 0 outside support
```

Trap / Warning

`np.interp` needs xp sorted ascending and by default *clamps* to edge values outside the range. Always pass `left=0.0, right=0.0` for finite-support signals. Never interpolate a genuinely discrete signal onto non-integer indices.

6 Even & Odd Decomposition

```
def odd_even_decomposition(x):
    xr = x[::-1]                # x[-n]  (symmetric axis)
    x_odd  = 0.5 * (x - xr)     # NO extra /2 !
    x_even = 0.5 * (x + xr)
    return x_odd, x_even        # check your required return ORDER
```

Trap / Warning

Off-by-a-factor trap: a stray extra `/2` on the odd term halves it. The instant check: `x_even + x_odd` must reconstruct `x` exactly (`np.allclose(xe+xo, x)`). If not, coefficients are wrong.

Tip

Verification lines graders reward: `allclose(xe, xe[::-1])` (even), `allclose(xo, -xo[::-1])` (odd), `allclose(xe+xo, x)` (reconstruction).

7 Energy & Power

```
# Discrete
E = np.sum(np.abs(x)**2)                # energy
P = np.sum(np.abs(x)**2) / len(x)       # average power

# Continuous (numerical integration)
E = np.trapezoid(np.abs(x)**2, t)       # integral of |x|^2 dt  (NumPy
    >= 2.0)
# E = np.trapz(...)  on NumPy < 2.0 (renamed to trapezoid in 2.0)
P = E / (t[-1] - t[0])
```

Tip

`np.abs` before squaring keeps the formula correct for complex signals too ($|x|^2 = x \cdot \text{conj}(x)$). **Version note:** `np.trapz` (NumPy < 2.0) was renamed `np.trapezoid` (NumPy ≥ 2.0).

8 Sinusoids: Time Shift vs Phase Shift

```
def sinusoid(n, A, W0, phi):
    return A * np.cos(W0 * n + phi)

def time_shift_sinusoid(n, A, W0, phi, n0):
    return A * np.cos(W0 * (n - n0) + phi)   # x[n-n0]

def phase_change_sinusoid(n, A, W0, phi, phi0):
    return A * np.cos(W0 * n + phi - phi0)   # state your sign
        convention

mse = lambda a, b: float(np.mean((a - b)**2))    # compare two signals
```

Tip

The key relationship: a time shift by n_0 equals a phase change of `phi0 = W0 * n0` (MSE $\approx$ 0). The reverse only matches exactly when `phi0 = W0` $\times$ integer — otherwise the best integer shift is an approximation (small nonzero MSE). Be ready to explain that asymmetry.

9 Linearity Test

```
def is_linear(system, x1, x2, a, b):
    lhs = system(a*x1 + b*x2)          # combine THEN process
    rhs = a*system(x1) + b*system(x2)  # process THEN combine
    return np.allclose(lhs, rhs)
```

Tip

Instant nonlinearity detector: feed the zero signal. A linear system must give $T\{0\} = 0$. If `system(np.zeros_like(x))` is nonzero (e.g., $x + 5$) $\rightarrow$ nonlinear, no further test needed.

Trap / Warning

Test complex coefficients too (e.g. `a=1+2j`) if conjugate / real-part systems appear — they pass for real a, b but fail for complex. Run several random trials, not one.

10 Time-Invariance Test

```
def delay(x, k):                          # zero-padded, non-circular
    if k > 0: return np.concatenate((np.zeros(k), x[:-k]))
    if k < 0: return np.concatenate((x[-k:], np.zeros(-k)))
    return x.copy()

def is_time_invariant(system, x, k):
    lhs = system(delay(x, k))     # shift THEN process
    rhs = delay(system(x), k)     # process THEN shift
    return np.allclose(lhs, rhs)
```

Trap / Warning

Suspect list: any bare `n` or `t` that appears *outside* the signal's own argument (like `n*x[n]` or `x(t)*cos(t)`) breaks time-invariance even though the system may be linear.

11 Causality Test (Perturb the Future)

```
def is_causal(system, x, n_trials=8, seed=0):
    rng = np.random.default_rng(seed)
    N = len(x)
    y0 = system(x)
    for _ in range(n_trials):
        n0 = rng.integers(0, N-1)          # a "now" index
        xp = x.copy()
        xp[n0+1:] += rng.normal(scale=10, size=N-n0-1) # scramble
            FUTURE only
        if not np.allclose(y0[:n0+1], system(xp)[:n0+1]):
            return False        # past/present changed -> peeked ahead
    return True
```

Tip

Analytical shortcut: if the formula uses any `x[n+k]` with $k > 0$ (future), it's non-causal. Centered averages, `x[n+1]`, and `x[-n]` are all non-causal. Backward-only filters are causal.

12 Matplotlib for the Lab

Discrete signals use `stem`; continuous use `plot`. Always label axes, add a legend, and save.

Discrete signal — stem plot

```
import matplotlib.pyplot as plt

n = np.arange(-8, 9)
plt.figure(figsize=(8, 3))
plt.stem(n, x)                  # stems at integer indices
plt.xticks(np.arange(-8, 9, 1))  # tick every integer
plt.ylim(-1, 3)
plt.title("x[n]")
plt.xlabel("n (Time Index)")
plt.ylabel("x[n]")
plt.grid(True)
plt.savefig("x[n].png")      # save BEFORE show
plt.show()
```

Continuous signal — line plot with legend

```
t = np.linspace(-4, 4, 4001)
plt.figure()
plt.plot(t, x,  label="x(t)")
plt.plot(t, xe, label="xe(t)")
plt.plot(t, xo, label="xo(t)")
plt.title("Even-Odd Decomposition")
plt.xlabel("t"); plt.ylabel("Amplitude")
plt.grid(True); plt.legend()
plt.show()
```

Tip

Save vs show order: call `savefig()` *before* `show()` — showing can clear the figure, leaving a blank file. If a template marks its plot helpers "do not modify," put a single `plt.show()` at the end of `main()` instead of editing them (one call displays all open figures).

Detailed Reference

R1: Array Creation

```
np.arange(start, stop, step=1, dtype=None)
```

Returns: 1-D array of values `start, start+step, ...` up to but **not including** stop.

```
np.arange(9)              # [0 1 2 3 4 5 6 7 8]  (one arg = stop, start
    =0)
np.arange(-8, 9)          # [-8 ... 8]  need 9 to include 8
np.arange(0, 10, 2)        # [0 2 4 6 8]  step of 2
np.arange(8, -1, -1)      # [8 7 6 5 4 3 2 1 0]  negative step counts
    down
```

Detail	Behavior
`dtype`	`int` args → `int64`; any `float` arg → `float64`
`stop`	always excluded (half-open interval)
`direction`	negative step counts downward

```
np.linspace(start, stop, num=50, endpoint=True, retstep=False,
dtype=None)
```

Returns: exactly `num` evenly spaced points; endpoint included by default.

```
np.linspace(0, 1, 5)                    # [0. 0.25 0.5 0.75 1.]
np.linspace(0, 1, 5, endpoint=False)  # [0. 0.2 0.4 0.6 0.8]
np.linspace(-4, 4, 4001)                # dense continuous-time axis
```

R2: Reorder & Join

```
x[::-1]       np.flip(m, axis=None)
```

Returns: the array with element order reversed. Slicing gives a *view* (cheap, no copy); flip returns a new array.

```
np.concatenate((a1, a2, ...), axis=0)
```

Returns: one array formed by joining the inputs along `axis`. Inputs go in a single tuple/list.

R3: Rounding & Type Conversion

```
np.round(a, decimals=0)       np.floor(a)       np.ceil(a)
np.trunc(a)
```

Returns: a float array with values rounded. `trunc` chops toward zero.

Trap / Warning

Banker's rounding: `np.round` rounds halves to the nearest *even* integer, so `round(0.5)=0`, `round(1.5)=2`, `round(2.5)=2`.

R4: Search & Conditional Select

```
np.searchsorted(a, v, side='left')
```

Returns: index/indices where each value in v would be inserted to keep sorted array a ordered. Binary search: $O(\log n)$.

```
np.where(condition, A, B)        np.where(condition)
```

Form 1: Vectorized if/else (pick A where True, else B). Form 2: Returns indices where True.

R5: `np.interp` — Linear Interpolation

```
np.interp(x, xp, fp, left=None, right=None, period=None)
```

Returns: interpolated value(s) at query points x, from known samples `(xp, fp)`.

Parameter	Meaning & Default
x	query point(s) — scalar or array
xp	known sample x-coords, **must be ascending**
fp	known sample y-values, same length as xp
`left`	value for $x < xp[0]$ — default `fp[0]` (clamp)
`right`	value for $x > xp[-1]$ — default `fp[-1]` (clamp)

R6: Reduce & Integrate

```
np.sum(a, axis=None, keepdims=False)        np.mean(a, axis=None)
```

```
np.sum(np.abs(x)**2)          # energy: reduce whole array to one number
np.mean((a - b)**2)       # MSE (mean squared error)
```

```
np.trapezoid(y, x=None, dx=1.0)
```

Returns: the trapezoidal-rule integral of y. Note: NumPy $<$ 2.0 calls this `np.trapz`.

R7: `np.allclose` — Float-Safe Equality

```
np.allclose(a, b, rtol=1e-05, atol=1e-08, equal_nan=False)
```

Returns: a single `bool`: True if every pair of elements is equal within tolerance. The verification workhorse for every property test.

Trap / Warning

Never use == on float results. Floating math produces tiny errors (e.g. `0.1+0.2 == 0.30000000000000004`), so exact equality reports False. Use `allclose`.

R8: Sequence Operations

```
np.diff(a, n=1, axis=-1, prepend=..., append=...)
```

Returns: differences $a[i] - a[i-1]$. Length shrinks by n unless you prepend/append.

```
np.cumsum(a, axis=None)
```

Returns: the cumulative sum. This is the accumulator system $y[n] = \sum_{k \le n} x[k]$.

R9: Random (for Property Tests)

```
rng = np.random.default_rng(seed=None)
```

```
rng = np.random.default_rng(0)          # fixed seed -> same stream
    every run
rng.normal(loc=0, scale=10, size=3)  # bell curve, mean 0, spread 10
rng.integers(low=0, high=10, size=4)  # ints in [0,10)  (high excluded)
```

R10: Indexing, Slicing & Boolean Masks

```
x[2:5]       # positions 2,3,4 (stop excluded)
x[::-1]      # reversed  (step = -1)
mask = np.abs(t) <= 1.0          # array of True/False
x[mask] = 1.0 - np.abs(t[mask])  # WRITE into those positions
valid = (idx >= -8) & (idx <= 8)
```

Tip

The +8 rule ($n = -8 \ldots 8$): array position = signal index + 8. So `x[n + 8]` reads the sample at signal index n. This offset appears in every discrete task — it converts a signal index into an array position.

R11: Function Quick-Reference

Function	What it does	Key gotcha
`np.arange(a,b,s)`	Integer/step axis	Stop excluded
`np.linspace(a,b,N)`	N points over range	Endpoint included
`np.zeros_like(a)`	Zeros, same shape	Copies dtype too
`x[::-1]` / `np.flip`	Reverse values	Also flip axis `-n[::-1]`
`np.concatenate((a,b))`	Join arrays	Pass one tuple/list
`np.roll(x,k)`	Circular shift	Wraps — not a real delay
`np.round / floor / ceil`	Rounding	`.astype(int)` to index
`np.searchsorted(a,v)`	Insertion position	Verify `a[pos]==v`
`np.clip(a,lo,hi)`	Bound values	Crash-guard for indexing
`np.where(cond,A,B)`	Elementwise select	Vectorized if/else
`np.interp(q,xp,fp)`	Linear interpolation	Set `left=0,right=0`
`np.sum / mean`	Reduce to scalar	Use `np.abs` for complex
`np.trapezoid(y,x)`	Numerical integral	`np.trapz` on NumPy $<$ 2.0
`np.allclose(a,b)`	Float-tolerant equality	Use instead of ==
`np.diff(x,prepend=0)`	Consecutive differences	Keeps length with prepend
`np.cumsum(x)`	Running sum	Accumulator system
`rng.normal / integers`	Random values	Seed for repeatability
`plt.stem(n,x)`	Discrete plot	For x[n]
`plt.plot(t,x)`	Line plot	For x(t)
`plt.savefig(...)`	Write image	Call before `show()`